

→ Squeeze

$$[1, 3, 5, 7, 9]$$

sliding window is
also two pointers

sliding window }

$L \rightarrow LR$
 $R \rightarrow$

Sliding Window

→ Array / String : Subarray, Substring

1) Variable size sliding window

Explained with LeetCode QNo.3: Longest Substring without Repeating Characters.
Ex: Input = "abcabab" Output = "abc"

Eg: Input = "abcabcbb"

Explanation: The answer is

Output = 3

"abc" with the length of 3

$$W = (R - L) + 1$$

#For sliding window
you always ask the
question "is our
window valid?"

$$W = (R - L) + 1$$

Set = {~~a~~ ~~b~~ ~~c~~ ~~a~~ ~~b~~ ~~c~~ ~~b~~}. ^{1) add} str[R] to ^{2) remove} the set ~~str[R] from~~ longest = ~~1 2 3~~

def lengthOfLongestSubstring(s: str) -> Int:

$$g=0$$

$\text{Jongest} = 0$

~~set = set()~~
$$n = \text{len}(s)$$

```
for r in range(n):
```

while $s[x]$ in set:

Sett. someore (802)

 $0.4 = 4$

Time : $O(n)$

Space: $O(n)$

$$\bullet W = (x-1) + 1$$
$$\text{longest} = \max(\text{longest}, w)$$
~~set.add(s)~~

Sietuon longest.

2) Fixed length Sliding Window

Explained with leetcode Qno 643 → Maximum Average Subarray

Eg: Input: $\text{nums} = [1, 12, -5, -6, 50, 3]$, $k=4$

Output: 12.75000

Explanation: Maximum average is $(12 - 5 - 6 + 50) / 4 = 12.75$

$k=4$

1 12 -5 -6 50 3
 1 2 3 4 5 6

$\text{max_avg} = -\infty$

$\text{cur_sum} += \text{nums}[i]$

$\text{cur_sum} = 0$ 13

$\text{cur_sum} -= \text{nums}[i-k]$

def findMaxAverage(self, nums: List[int], k: int) → float:

$n = \text{len}(\text{nums})$

$\text{cur_sum} = 0$

for i in range(k):

$\text{cur_sum} += \text{nums}[i]$

$\text{max_avg} = \text{cur_sum} / k$

for i in range(k, n):

$\text{cur_sum} += \text{nums}[i]$

$\text{cur_sum} -= \text{nums}[i-k]$

$\text{avg} = \text{cur_sum} / k$

$\text{max_avg} = \max(\text{max_avg}, \text{avg})$

return max_avg

Time: $O(n)$

Space: $O(1)$